

# What It Actually Takes to Run 36 AI Agents in Production

Nyquist Research

11 May 2026

## Abstract

Spinning up agents is easy. Keeping them synchronized, isolated from each other's failures, fed with consistent real-time data, responsive under load, compliant with regulatory obligations, and economically viable given clearing and connectivity costs — that is a different engineering problem. One we have had to solve.

*Published: 11 May 2026   Author: Nyquist Research   Reading time: 22 min   Topic: Infrastructure · MLOps   Stack: vLLM · Kafka · Redis · PTP · OTel*

## Compute topology · three speed tiers before writing a single line

Three tiers. Different hardware. Different stacks. Different containerization rules.

Treating all agents as equivalent compute is the single most common — and most expensive — infrastructure mistake.

**T1 · HOT <10 ms** Bare-metal critical path. Market data ingestion, order routing, position tracking. Pinned cores, ring buffers, zero GC pressure.

- Stack: C++ / Rust
- Container: no — bare process
- Hardware: co-lo, 10GbE
- Cost / mo: \$800 – \$2,000

**T2 · WARM 10 – 500 ms** Where the AI agents live. LLM inference, signal generation, risk reasoning. Weights pinned in VRAM; cold-start is unacceptable.

- Stack: vLLM / TGI · FastAPI
- Container: Docker / K8s
- Hardware: A10G / A100, dedicated VRAM
- Cost / mo: \$2,800 – \$4,200

**T3 · COLD >500 ms** Stateless, batch, cost-optimized. Backtesting, model retraining, compliance reporting. Spot instances, horizontal scale.

- Stack: Ray / Dask / Celery
- Container: stateless
- Hardware: Spot · AWS EC2 / Railway

- Cost / mo: \$300 – \$800

*Latency asymmetry forces segmentation. If you do not tier first, you over-engineer cold agents or under-serve hot ones. 36 agents · 3 tiers · one institutional stack.*

Spinning up 36 AI agents is easy. Keeping them synchronized, isolated from each other's failures, fed with consistent real-time data, responsive under load, compliant with regulatory obligations, and economically viable given clearing and connectivity costs — **that's a different engineering problem entirely**. One we've had to solve in production.

## 1 Why agent infrastructure differs from standard MLOps.

Most MLOps frameworks assume stateless, batch-oriented workloads. **Financial AI agents break every one of those assumptions.**

### Property 01 · Stateful agents

Recovery is a state-replay operation — not a pod restart.

A trading agent isn't just a compute process — it holds open positions, pending orders, and a live market context window. When it crashes, you don't lose a task. **You inherit a risk event:** positions that are unhedged, orders that may or may not have hit the exchange, and a gap in your audit trail. *Recovery isn't `kubectl restart pod`; it's a structured state-replay operation with verification.*

### Property 02 · Temporal consistency

80ms of clock skew is a wrong-way risk generator.

If Agent A is pricing a spread on a data snapshot 80ms older than Agent B's, they produce divergent signals on correlated instruments. At 36 agents running concurrently, clock skew isn't a data quality issue — **it's a systematic wrong-way risk generator.**

### Property 03 · Cascading failures

Failures propagate faster than human oversight.

Agent A generates a bad signal → Agent B acts on it → Agent C hedges the wrong position. The failure propagates through your fleet in milliseconds. *By the time a human sees the alert, you've already traded.* Circuit breakers must live in the fleet, not in the operator's chair.

### Property 04 · Latency asymmetry

LLM inference (50–500ms) is incompatible with HFT.

LLM inference latency is categorically incompatible with HFT but perfectly acceptable for execution agents in medium-frequency strategies. **Segment your fleet by speed tier before writing a single line of agent code** — or you will either over-engineer cold-path agents or under-serve latency-critical ones.

## Property 05 · Auditability

Every agent decision logged with input/output hash, model version, `timestamp_ns`.

Every agent decision must be logged with: `agent_id`, `model_version`, `input_hash`, `output_action`, `confidence_score`, `timestamp_ns`. *Not for post-mortem debugging — for compliance review.* CAT reporting, MiFID II trade reconstruction, and FIA audit obligations require this from day one — not as a retrofit.

## 2 Layer 1 · Compute topology.

The single most common infrastructure mistake is treating all agents as equivalent compute workloads. **They are not.** The correct mental model is *three distinct speed tiers*, each with different hardware, containerization, and stack requirements.

Table 1: Compute tier summary

| Tier          | Latency SLA | Hardware                                     | Containerized?    | Cost / month      |
|---------------|-------------|----------------------------------------------|-------------------|-------------------|
| Tier 1 · Hot  | <10 ms      | Co-located bare metal · pinned cores · 10GbE | no — bare process | \$800 – \$2,000   |
| Tier 2 · Warm | 10 – 500 ms | GPU cloud (A10G / A100) · dedicated VRAM     | Docker / K8s      | \$2,800 – \$4,200 |
| Tier 3 · Cold | >500 ms     | Spot instances (AWS EC2 · Railway)           | stateless         | \$300 – \$800     |

### 2.1 Tier 1 · the bare-metal critical path.

Handles market data ingestion, order routing, and position tracking. **No containerization overhead** — bare processes with CPU affinity pinned to isolated cores, pre-allocated ring buffers, and zero GC pressure. The stack is C++ or Rust for the critical path; Python appears only in orchestration glue, *never in the latency-sensitive loop*.

### 2.2 Tier 2 · where the agents live.

Model weights must be loaded into GPU VRAM permanently — cold-start on model load is production-unacceptable. **vLLM or TGI** serves inference; FastAPI exposes the internal agent API layer. Key management note: vLLM’s KV-cache pre-allocates GPU memory using `gpu_memory_utilization` — *PagedAttention reduces fragmentation waste from 60–80% down to under 4%*, giving you 2–4× throughput on the same hardware. This matters at 36 agents generating sustained inference load.

### 2.3 Tier 3 · stateless batch, cost-optimized.

Backtesting, model retraining, and compliance reporting. Stateless, horizontally scalable, cost-optimized on Spot instances. Ray, Dask, or Celery for distributed compute. *Workloads here can tolerate preemption; place nothing here that can’t be retried.*

### 3 Layer 2 · Agent orchestration.

At 36 concurrent agents, orchestration is not a feature — **it's a discipline**. Start with a taxonomy.

#### Agent taxonomy · five archetypes

##### Data agents

Market data normalization · feed validation · sequence gap detection.

##### Signal agents

Alpha generation · sentiment scoring · factor construction.

##### Risk agents

Position limit monitoring · real-time VaR breach detection · margin status.

##### Execution agents

Order routing · TCA · slippage attribution.

##### Meta-agents

Fleet-level watchdogs · circuit breakers · cross-agent correlation monitoring.

#### 3.1 Isolation model.

Each agent runs in its own container with explicit resource limits: CPU quota, memory ceiling, GPU allocation. At 36 agents on shared infrastructure, the **noisy-neighbor problem is not theoretical** — one agent with a memory leak (vLLM KV-cache growth under sustained load is a documented failure mode) will degrade adjacent agents silently before it crashes.

#### 3.2 Communication topology.

- **Pub/Sub via Kafka 3.x or Redis Streams** for market data fan-out — one canonical bus, N subscribers.
- **gRPC** for synchronous agent-to-agent calls where latency and contract enforcement matter.
- **Redis 7.x** as the shared state store for position and risk state — *your single source of truth; agent-local state is a consistency liability*.

#### 3.3 Circuit breaker pattern.

Every agent has a watchdog process. If an agent misses N consecutive heartbeats, or if its output deviates anomalously (position size  $>3\sigma$  from historical distribution, suspiciously uniform confidence scores suggesting model collapse), it is **automatically isolated and flagged for human review** before it can affect the rest of the fleet.

Below is a minimal agent health monitor implementing this pattern using **asyncio + Redis**:

*Python · asyncio + redis heartbeat watchdog · circuit breaker*

```
import asyncio
import redis.asyncio as aioredis
```

```

from datetime import datetime, timezone

HEARTBEAT_TTL          = 10          # seconds before agent considered stale
CHECK_INTERVAL         = 5
CIRCUIT_BREAK_THRESHOLD = 3          # consecutive misses before isolation

async def monitor_fleet(agent_ids: list[str],
                        redis_url: str = "redis://localhost:6379"):
    r = await aioredis.from_url(redis_url, decode_responses=True)
    miss_counts: dict[str, int] = {aid: 0 for aid in agent_ids}

    while True:
        await asyncio.sleep(CHECK_INTERVAL)
        now = datetime.now(timezone.utc).timestamp()

        for agent_id in agent_ids:
            last_beat = await r.get(f"heartbeat:{agent_id}")

            if last_beat is None or (now - float(last_beat)) > HEARTBEAT_TTL:
                miss_counts[agent_id] += 1
                print(f"[{datetime.utcnow().isoformat()}] WARN: {agent_id} "
                      f"missed heartbeat "
                      f"({miss_counts[agent_id]}/{CIRCUIT_BREAK_THRESHOLD})")

                if miss_counts[agent_id] >= CIRCUIT_BREAK_THRESHOLD:
                    await r.set(f"circuit_breaker:{agent_id}", "OPEN")
                    await r.publish("fleet_alerts", f"CIRCUIT_OPEN:{agent_id}")
                    print(f"[CIRCUIT BREAKER] {agent_id} isolated "
                          f"-- flagged for review")
            else:
                miss_counts[agent_id] = 0
                await r.set(f"circuit_breaker:{agent_id}", "CLOSED")

```

**Pattern guarantees:** three consecutive missed beats triggers `circuit_breaker:<agent_id>` = OPEN and publishes to `fleet_alerts`. Meta-agents subscribe to this channel and refuse to route new work to the isolated agent until human-acknowledged.

## 4 Layer 3 · Data consistency.

This is the **most chronically underengineered** component in agent infrastructure. Teams invest heavily in compute and orchestration, then let agents fetch their own data independently. *This is how you get fleet-level temporal inconsistency.*

### Non-negotiable design decisions

#### Single data bus

All 36 agents subscribe to **one canonical stream**. Independent fetching creates fan-out inconsistency: agents on different fetch cycles, different staleness profiles, producing correlated-but-inconsistent signals.

### Sequence numbers

Every message carries a sequence number. Agents validate in-order processing. **Gap detection is mandatory** — a missing sequence triggers replay, not silent skip.

### Snapshot + delta

On restart, replay from last checkpoint and apply deltas forward. **Starting from empty state on crash recovery is unacceptable** — your restarted agent would operate with zero market context while the rest of the fleet has hours of accumulated state.

### Time sync

NTP achieves only millisecond accuracy — insufficient for multi-agent coordination under load. **Use PTP (IEEE 1588)** on co-located infrastructure (sub-microsecond accuracy) and **Lamport logical clocks** for cloud-distributed deployments where PTP hardware is unavailable.

### Data validation

Every incoming tick validated against price bounds ( $\pm N\sigma$  from VWAP), sequence continuity, and timestamp monotonicity **before reaching any agent**. Bad data rejected at the bus — not inside agent logic.

## 5 Layer 4 · Observability.

You cannot debug a 36-agent fleet by reading logs sequentially. **You need structured observability across three dimensions.**

Table 2: Observability alert matrix

| Metric                        | Tool                 | Alert threshold   | Response playbook                                                          |
|-------------------------------|----------------------|-------------------|----------------------------------------------------------------------------|
| Agent inference latency p99   | Prometheus + Grafana | >500 ms sustained | Scale GPU tier · check KV-cache saturation                                 |
| Fleet health (active / total) | Custom exporter      | <80% healthy      | Page on-call · isolate failed agents · check Redis connectivity            |
| Data bus lag                  | Kafka consumer lag   | >100 ms           | Escalate to data engineering · check broker throughput                     |
| Risk agent heartbeat          | Redis TTL monitor    | silent >30 s      | <i>Immediate escalation</i> — risk-agent silence is a trading-halt trigger |
| Decision log throughput       | Loki / ELK           | drop to zero      | Audit-trail gap = <i>regulatory exposure</i> · halt new agent decisions    |
| GPU memory utilization        | DCGM + Grafana       | >90% VRAM         | Investigate KV-cache leak · roll inference server                          |

### 5.1 Metrics · time-series.

Prometheus + Grafana. **Per-agent:** inference latency p50/p95/p99, tokens/sec, queue depth, error rate, position delta generated per minute.

## 5.2 Logs · structured.

Every agent decision emitted as a JSON document: `agent_id`, `model_version`, `input_hash`, `output_action`, `confidence_score`, `timestamp_ns`. Stored in Loki or ELK. *Not optional* — *it's the raw material for post-trade analysis and regulatory audit response*.

## 5.3 Traces · distributed.

OpenTelemetry spans covering the full signal path — *market data tick* → *agent inference* → *risk check* → *order generation* — as a single distributed trace with per-span timing. **When something goes wrong at 09:32 on a high-volatility open, you need millisecond-resolution attribution, not grep.**

## 6 Cost reality · compute & data infrastructure.

The visible cost stack. The first table any infrastructure proposal includes — and the one that *understates* the true institutional bill by a factor of 3–5×

Table 3: Compute & data infrastructure costs

| Component                      | Spec                            | Cost / month                |
|--------------------------------|---------------------------------|-----------------------------|
| Co-lo / bare metal (Tier 1)    | 2× dedicated servers · 10GbE    | \$800 – \$2,000             |
| GPU inference (Tier 2)         | 4× A10G (AWS g5.2xlarge)        | \$2,800 – \$4,200           |
| Orchestration cluster          | 3-node K8s · 8 vCPU · 32GB each | \$600 – \$900               |
| Data bus (Kafka managed)       | Confluent Cloud / self-hosted   | \$400 – \$800               |
| Observability stack            | Grafana Cloud / self-hosted     | \$200 – \$500               |
| Market data feeds              | Asset class dependent           | \$500 – \$5,000+            |
| Cold compute (batch)           | Spot instances · variable       | \$300 – \$800               |
| <b>Total · excl. Bloomberg</b> |                                 | <b>~\$5,000 – \$9,000</b>   |
| <b>Total · incl. Bloomberg</b> |                                 | <b>~\$27,000 – \$33,000</b> |

A practical note on LLM inference economics: at **36 agents** × **1,000 inference calls/day**, API costs (GPT-4o, Claude 3.5 Sonnet) exceed self-hosted GPU costs (Llama 3.1 70B on A10G) at *approximately the 3-month mark*. Self-hosted inference requires dedicated MLOps overhead — model updates, GGUF/AWQ quantization for VRAM optimization, vLLM version management — but at fleet scale the economics are straightforward.

## 7 The hidden cost stack — clearing, connectivity, compliance.

Most infrastructure breakdowns stop at compute and data. **The real institutional cost stack includes three additional layers that are rarely discussed publicly — and which dwarf cloud bills for any serious trading operation.**

### 7.1 Clearing & settlement.

Key structural realities: DTC/NSCC membership is not available to most fintech startups — **clearing broker dependency introduces counterparty risk that must be explicitly**

Table 4: Clearing &amp; settlement costs by asset class

| Asset class              | Clearing model          | Cost range                           | Min. capital      |
|--------------------------|-------------------------|--------------------------------------|-------------------|
| US Equities              | PB / self-clear         | \$0.001 – \$0.005 / share            | \$500K+           |
| US Futures<br>(CME/CBOT) | FCM + CME clearing      | \$0.10 – \$0.35 / contract<br>/ side | \$100K+ margin    |
| US Options               | OCC via prime broker    | \$0.02 – \$0.05 / contract           | \$50K+            |
| Crypto · CEX             | Exchange native         | 0 – 10 bps                           | none              |
| Crypto · CeDeFi          | Smart contract + bridge | gas + 5 – 50 bps                     | none              |
| FX Spot                  | Prime broker            | 0.5 – 2 pips                         | \$1M+ credit line |

**modeled.** SPAN initial margin for a 36-agent futures fleet can reach \$500K–\$5M depending on gross notional. Variation margin requires a dedicated liquidity buffer of 10–15% of gross notional available as daily cash settlement. *Short locate fees on hard-to-borrow names can reach 50%+ annualized* — these must be modeled as explicit strategy costs, not broker footnotes.

## 7.2 Market connectivity & co-location.

Table 5: Market connectivity options and costs

| Connectivity                        | Latency           | Monthly cost       | Best for             |
|-------------------------------------|-------------------|--------------------|----------------------|
| Co-lo bare metal (same rack)        | 1 – 10 $\mu$ s    | \$8,000 – \$20,000 | HFT · market making  |
| Co-lo bare metal<br>(cross-connect) | 10 – 100 $\mu$ s  | \$3,000 – \$8,000  | Stat arb · futures   |
| Managed extranet (TNS / Colt)       | 500 $\mu$ s – 2ms | \$2,000 – \$10,000 | Mid-frequency        |
| Cloud near co-lo (AWS / Azure)      | 1 – 5 ms          | \$500 – \$2,000    | Research · swing     |
| Broker DMA (FIX)                    | 2 – 10 ms         | \$0 – \$500        | Low-frequency · algo |

NYSE co-location at Mahwah (NJ) and NASDAQ at Carteret (NJ) run **\$5,000–\$15,000/month per cabinet** plus cross-connect fees. Direct market data — NYSE Integrated Feed (~\$3,000/mo), NASDAQ TotalView-ITCH (~\$1,500/mo), CME Globex data (\$1,500–\$6,000/mo by product group) — adds significantly beyond cloud bills. *FIX connectivity via executing broker introduces 1–5ms of additional latency that is acceptable for medium-frequency strategies but categorically unacceptable for HFT.*

## 7.3 Compliance & regulatory infrastructure.

**CAT reporting** (US equities/options, broker-dealer obligation) has seen actual build-out costs reach ~\$500M industry-wide, with annual operations running ~\$200M/year across the ecosystem. *Per-firm:* \$50,000–\$200,000 initial setup, \$10,000–\$50,000/year ongoing. **EMIR / MiFID II ARM** connectivity for EU-regulated entities runs \$30,000–\$100,000/year. **AML/KYC for crypto** — Chainalysis KYT, Elliptic, or equivalent — adds \$20,000–\$150,000/year. The **FIA-compliant kill switch** — sub-100ms full fleet shutdown with hardware-level enforcement — is a \$5,000–\$20,000 one-time engineering and testing investment that is *not optional* for any FCM-connected operation.

## 8 Full cost stack.

Adding everything that gets billed monthly when you go live at institutional scale:

Table 6: Full institutional cost stack

| Cost category                   | Low / mo         | High / mo        | Notes                           |
|---------------------------------|------------------|------------------|---------------------------------|
| Compute (Tier 1–3)              | \$5,000          | \$9,000          | excl. Bloomberg                 |
| Market data feeds               | \$3,000          | \$15,000         | asset class dependent           |
| Exchange connectivity           | \$3,000          | \$20,000         | DMA vs. managed                 |
| Clearing & PB fees              | \$2,000          | \$20,000         | volume dependent                |
| Margin reserve (capital)        | \$50,000         | \$500,000        | capital, not OpEx               |
| Trade surveillance (CAT/EMIR)   | \$4,000          | \$17,000         | amortized annual                |
| AML/KYC (crypto)                | \$2,500          | \$12,500         | amortized annual                |
| Kill switch + pre-trade         | \$500            | \$2,000          | amortized                       |
| Audit log storage               | \$100            | \$200            | S3 Glacier                      |
| <b>Monthly OpEx total</b>       | <b>~\$20,000</b> | <b>~\$95,000</b> | <b>excl. capital</b>            |
| <b>Annual + capital reserve</b> | <b>~\$290K</b>   | <b>~\$2.2M</b>   | <b>full institutional stack</b> |

The fixed cost base of a full institutional stack is irrational at small scale. What changes the equation is *shared infrastructure* where the compliance, connectivity, and data layers are pre-built and already amortized.

## 9 Break-even analysis.

Rule of thumb: **full-stack annual cost of \$350K–\$1M, against a management fee of 1–2% of AUM, implies a break-even between \$17.5M and \$100M AUM** before the infrastructure investment is economically justified.

*Python · break-even table scenario matrix · mgmt-fee only*

```
annual_costs = {"low": 350_000, "mid": 650_000, "high": 1_000_000}
mgmt_fees     = {"1.0%": 0.010, "1.5%": 0.015, "2.0%": 0.020}

header = f"{'Scenario':<25}" + "".join(f" {k:>12}" for k in mgmt_fees)
print(header)
print("-" * (25 + len(mgmt_fees) * 14))

for cost_label, annual_cost in annual_costs.items():
    row = f"{cost_label + ' ($' + str(annual_cost//1000) + 'K/yr)':<25}"
    for fee_label, fee_rate in mgmt_fees.items():
        breakeven_aum = annual_cost / fee_rate
        row += f" ${breakeven_aum/1e6:>9.1f}M"
    print(row)

print("\nNote: performance fee inclusion reduces break-even AUM by 30–50%"
      " at Sharpe > 1.0.")
```

**What the matrix encodes:** at 1.0% mgmt fee + \$1M/yr full institutional stack, you need \$100M AUM to clear the bar. Drop to 2% + low-cost stack and you clear at \$17.5M. *Performance fee economics shift this materially when Sharpe >1.0.*

## 9.1 Sensitivity matrix (break-even AUM).

Table 7: Break-even AUM sensitivity matrix

| Cost scenario      | 1.0% mgmt fee | 1.5% mgmt fee | 2.0% mgmt fee |
|--------------------|---------------|---------------|---------------|
| Low · \$350K / yr  | \$35.0M       | \$23.3M       | \$17.5M       |
| Mid · \$650K / yr  | \$65.0M       | \$43.3M       | \$32.5M       |
| High · \$1.0M / yr | \$100.0M      | \$66.7M       | \$50.0M       |

## 10 The build vs. buy decision.

Two scenarios. **Architecture matters; AUM matters more.** The fixed cost base of a full institutional stack is irrational below a threshold — managed platforms with bundled clearing/connectivity/compliance are the economic answer at small scale, full-stack ownership is the answer past it.

### Build full stack when

*You control your own latency budget.*

- Strategy requires sub-10ms agent response time
- Proprietary model architecture unavailable via API
- Regulatory requires on-premise or private cloud deployment
- Fleet scale >50 agents makes API inference costs economically irrational
- AUM >\$50M justifies the fixed cost base

### Use managed platform when

*Research velocity is the binding constraint.*

- Team <5 engineers — infrastructure overhead kills research velocity
- Prototyping phase — validate strategy alpha before committing infrastructure
- Non-latency-critical use cases (research, reporting, compliance monitoring)
- AUM <\$20M — the fixed cost base is economically irrational

The Nyquist position: we built Tier 1 and Tier 2 ourselves and expose it as a managed API layer — including identity-resolved market data, normalized clearing-cost models, and pre-wired

observability. *Quant teams should not be spending engineering cycles on Kafka consumer-group rebalancing, GPU memory management, or EMIR reporting plumbing. They should be building strategies.*

Our infrastructure layer answers the question: “*what would it cost a 3-person quant team to replicate this?*” — **18–24 months and \$300K+ before writing a single line of alpha code.**

## 11 Failure modes nobody warns you about.

### F1 · Context window drift.

Long-running agents accumulate stale context across a trading session. Their decisions degrade progressively *without any visible error signal*. **Mitigation:** periodic context window resets keyed to market session boundaries, not calendar time.

### F2 · Model version skew.

During a rolling deployment, agents running different model checkpoints produce inconsistent signals on the same input. At the fleet level, this creates *spurious divergence that looks like genuine disagreement but is an artifact of deployment sequencing*. **Mitigation:** atomic fleet-level model version pinning with a synchronization gate before any agent produces output.

### F3 · Redis hot-key contention.

All 36 agents writing position updates to a shared key at market open creates throughput collapse. A 09:30 write storm is **entirely predictable and entirely avoidable** — but only if you’ve thought about key sharding before going live. **Mitigation:** per-agent namespaced keys with a meta-agent aggregating fleet state asynchronously.

### F4 · GPU memory leak in inference server.

vLLM’s KV-cache grows unbounded under sustained load without explicit eviction tuning — *OOM after 4–6 hours of production load*, with silent performance degradation (p99 latency inflation) for hours before the crash. **Mitigation:** explicit `max_num_seqs` bounds, Prometheus VRAM monitoring with alert at 90% utilization, scheduled inference server restarts during low-activity windows.

### F5 · Clock skew amplification.

50ms NTP drift across cloud regions causes agents to disagree on whether a candle belongs to timestamp T or T-1. The result is *correlated wrong-way hedges that appear as strategy misbehavior but are infrastructure failures*. **Mitigation:** PTP on co-located infrastructure (sub-microsecond accuracy), Lamport logical clocks for cloud-distributed deployments.

### F6 · Margin call cascade.

Multiple agents simultaneously breach position limits under flash volatility. Automatic de-risking orders arrive at the clearing broker simultaneously. Liquidity impact amplifies losses. *Margin*

*spiral*. **Mitigation:** fleet-level gross notional cap enforced at the meta-agent layer — **not per-agent layer**. Individual agent position limits are insufficient when 36 agents can simultaneously hit them in the same direction.

## 12 Key takeaways.

*Six rules · architecture before code*

1. **Segment agents by speed tier before writing a single line of code.** Tier 1/2/3 topology is an architectural commitment, not a deployment optimization. Retrofit cost is multiples of greenfield cost — and you may discover you need it the wrong way around (a Tier 2 strategy on Tier 3 economics).
2. **Single market data bus with sequence numbers is non-negotiable.** Independent data fetching at agent level is a consistency liability that compounds at fleet scale. *One bus, N subscribers, gap detection, replay on miss.*
3. **Model clearing and connectivity costs explicitly.** At any serious trading scale, they *exceed* compute costs. Ignoring them produces systematically incorrect P&L attribution — your “alpha” disappears into a fee structure you never modelled.
4. **Break-even AUM for full institutional stack is \$17.5M–\$100M.** Below this range, managed platforms with bundled infrastructure are economically rational. Above it, ownership of the stack becomes a moat.
5. **Kill switch and pre-trade controls are engineering problems, not compliance checkboxes.** FIA-compliant sub-100ms fleet shutdown is an infrastructure specification with hardware-level enforcement — not a policy document.
6. **Observability at agent level (p99 latency, structured decision logs, distributed traces) is the difference between debugging in minutes and debugging in days.** Retrofit is far more expensive than greenfield instrumentation. *Build the metrics, logs, and traces before the first agent goes live.*

## 13 Inputs & constraints.

### Inputs & assumptions

#### Agent count

**36 concurrent** production agents — mixed LLM, rule-based, ML model.

#### Infrastructure baseline

Hybrid cloud + co-located Tier 1.

#### LLM models

Llama 3.1 70B (self-hosted via vLLM) · GPT-4o (API fallback) · Claude 3.5 Sonnet (research agents).

#### Orchestration stack

Kubernetes 1.29+ · Kafka 3.x · Redis 7.x.

#### Observability

Prometheus · Grafana · OpenTelemetry · Loki.

## Clearing

CME · OCC · DTC/NSCC fee schedules 2025–2026.

## Connectivity

NYSE Mahwah · NASDAQ Carteret co-lo pricing, May 2026.

## Compliance

Nasdaq Surveillance · Chainalysis · Notabene.

## Cloud pricing

AWS us-east-1 on-demand, May 2026.

## Market coverage

US equities + futures + options + crypto.

### 13.1 Constraints & limitations.

- Cost estimates vary materially by region, reserved-instance discounts (up to 60%), and actual utilization patterns.
- H100 spot availability remains constrained with high preemption rates; **A10G / A100 on-demand** is more reliable for production Tier 2 workloads.
- Self-hosted LLM inference requires dedicated MLOps cycles — model updates, quantization (GGUF / AWQ), VRAM optimization — not captured in raw GPU cost estimates.
- Kubernetes adds operational complexity; for teams under 3 engineers, managed alternatives (Railway, Modal, Fly.io) may be more practical for Tier 2 workloads.
- Clearing cost estimates assume US-regulated structure; EU (EMIR), UK (FCA), and crypto-native structures differ materially.
- CAT reporting and EMIR obligations apply to *registered broker-dealers and FCMs*; unregistered entities face structurally different compliance frameworks.
- Break-even AUM analysis assumes flat fee structures — performance fee economics change the calculus significantly in favor of building.

## 14 Closing.

Spinning up 36 agents in a notebook takes an afternoon. Running them in production for a year — synchronized, audited, isolated, observable, and economically rational — takes 18–24 months and the better part of \$300K before a single line of strategy code ships. **Most of that cost is invisible until you’re trying to ship.** By then it’s a refactor, not a roadmap.

The infrastructure problem of 2026 is not “can we deploy LLM agents” — it’s “can we deploy them without rebuilding the institutional stack from scratch?”

**How does your team currently model clearing and connectivity costs against compute infrastructure?** We’re building a comprehensive cost model for institutional AI agent stacks and would value your real numbers.

If you’re designing infrastructure for AI agent fleets in financial contexts — or hitting these failure modes or cost surprises in your current stack — [we’d like to compare architecture and cost notes](#). No sales pitch.